

Interview cheat sheet: all lessons

Every lesson's cheat sheet in one place. ♦ = my addition beyond the video; everything else is from the lessons.

01 · What Is Inference & Inference Engineering?

[Full notes](#)

Say it in 30 seconds

Inference is running a trained model token by token for every request, so fleet-wide it costs more than training. Decode is memory-bandwidth-bound: every step streams all the weights to make one token per sequence. The core trade is latency vs throughput vs cost, and batching is lever one. The job: measure, find the bottleneck, change one thing, repeat.

Only numbers worth memorizing

- **H100:** 3.35 TB/s HBM, ♦ ~989 TFLOPS BF16. **Llama 3 8B:** 16 GB in BF16, ♦ ~128 KB KV per token.
- **Software matters:** batching gives ~15× cheaper tokens; vLLM gets up to 24× naive HF throughput.

Derive, don't memorize

Let P = parameters, B = batch size, BW = memory bandwidth.

1. Decode speed = bandwidth ÷ bytes per step

Every decode step streams all the weights once, so time per step is bytes moved over bytes per second.

$$t_{\text{step}} \approx \frac{2P \text{ bytes}}{BW} = \frac{16 \text{ GB}}{3.35 \text{ TB/s}} \approx 4.8 \text{ ms} \Rightarrow \leq 200 \text{ tok/s per user}$$

Halve the bytes (8-bit weights) and the ceiling doubles. That's why precision is a speed lever.

2. Memory-bound or compute-bound? Compare FLOPs per byte to the ridge ♦

Each weight does a multiply and an add (2 FLOPs) per token and costs 2 bytes to read. One read is shared by all B sequences:

$$\frac{\text{FLOPs}}{\text{byte}} \approx \frac{2P \cdot B}{2P} = B \quad \text{vs} \quad \text{ridge} = \frac{989 \text{ TFLOPS}}{3.35 \text{ TB/s}} \approx 300$$

Batch 1 gives ~1, far below 300, so decode is memory-bound and batching is nearly free speed. Prefill puts ~500 prompt tokens through one read, above the ridge, so it's compute-bound:

$$t_{\text{prefill}} \approx \frac{N_{\text{prompt}} \cdot 2P}{\text{peak FLOPS}} = \frac{500 \times 16 \text{ GFLOP}}{989 \text{ TFLOPS}} \approx 8 \text{ ms}$$

3. Cost = what you pay per hour ÷ what you make per hour

$$\frac{\$}{1\text{M tokens}} = \frac{\$ \text{ per GPU hour}}{\text{tok/s} \times 3600} \times 10^6 \quad \frac{3}{200 \times 3600} \times 10^6 \approx 4.17 \quad \frac{3}{3000 \times 3600} \times 10^6 \approx 0.28$$

4. Why batching isn't free: every sequence brings its own KV cache ♦

Each layer stores one K and one V vector per KV head for every past token:

$$\text{KV/token} = 2 \times L \times H_{\text{kv}} \times d_{\text{head}} \times \text{bytes} = 2 \times 32 \times 8 \times 128 \times 2 = 128 \text{ KB}$$

A step reads the weights once plus every sequence's cache (context length C), so steps slow down as the batch grows:

$$t_{\text{step}}(B) \approx \frac{2P + B \cdot C \cdot \text{KV/token}}{\text{BW}}$$

Prefill vs decode

	Prefill	Decode
Work	Whole prompt in one parallel pass	One token per sequence per step, serial
Bottleneck	Compute (FLOPs)	Memory bandwidth (bytes)
User sees it as	TTFT	Gap between tokens (ITL / TPOT)
Grows with	Prompt length	Output length, and batch × context (KV reads)

Symptom → lever (for "how would you fix it?" questions)

Symptom	Likely cause	First levers
High \$/token, GPU underused	Batch too small	Scheduling: batch more requests per step
Batch capped by out-of-memory	KV cache fills memory	Memory: pack KV tightly, reuse shared prefixes
Slow tokens even at low load	Too many weight bytes per step	Precision: 8/4-bit weights
High TTFT	Long prompts or queueing	Shorter prompts, prefix reuse, more replicas
Model doesn't fit one GPU	Size	Parallelism across GPUs, or precision

Rapid-fire Q&A

Question	Crisp answer
Why does batching help, and what does it cost?	One weight read serves all B sequences. The cost is KV memory and slower steps (derivation 4).
♦ Max tok/s for 70B BF16 on 8×H100?	Derivation 1: $140 \text{ GB} \div (8 \times 3.35 \text{ TB/s}) \approx 5.2 \text{ ms/step}$, so $\leq \sim 190 \text{ tok/s}$ before communication overhead.
Cost too high: where do you start?	Measure (utilization, batch size, TTFT/ITL) before changing anything; then check off-GPU wins such as a smaller model, shorter prompts, caching.

Traps to avoid

- "Just call `generate()`" or "buy a faster GPU": both ignore the scheduler, memory and setup.
- Trading away quality: a faster but worse answer is not a win.